System Documentation & Project Proposal

Task 1 — Planning & Management

Group No: G#01 • **Project Name:** TechXpress • **Submission Date:** 16/10/2026

1.1 Project Proposal & Executive Summary

TechXpress is a scalable, secure, and fully functional E-commerce web application engineered for browsing, purchasing, and managing electronic devices (e.g., laptops, smartphones, accessories). The platform leverages modern enterprise software design principles including N-Tier Architecture, Repository Pattern, Unit of Work, and Dependency Injection within the ASP.NET Core MVC framework to ensure strict separation of concerns, high maintainability, and data consistency.

Core Objectives:

- **Intuitive Shopping Experience:** Deliver an interactive user experience with real-time dynamic search, category filtering, and responsive UI.
- **Shopping Cart & Payments:** Provide robust session-based shopping cart handling and secure payment processing through Stripe API integration.
- **Access Control:** Implement Role-Based Access Control (RBAC) via ASP.NET Identity to manage Customer, Admin, and Delivery workflows.
- **Data Layer & Architecture:** Establish a scalable data abstraction layer using Entity Framework Core for transparent database operations and complete N-Tier decoupling.

1.2 Scope Statement

In Scope:

#	In Scope
1	Dynamic Product Browsing, Advanced Filtering, and Search by Categories
2	Shopping Cart Management with Session State Persistence
3	Secure Online Payment Integration via Stripe API
4	User Authentication & Role-Based Access Control (Customers, Admins, Delivery Personnel) via ASP.NET Core Identity

#	In Scope
5	Administration Dashboard (DataTables) for Full Product & Order Management
6	Live Order Tracking and Status Updates for Customers and Delivery Personnel

Out of Scope:

#	Out of Scope
1	Live Customer Support Chatbot or AI Virtual Assistant
2	Multi-currency and Multi-language Localization Support
3	Native Mobile Application (Restricted to Responsive Web Design)

Key Deliverables:

ID	Deliverable	Acceptance Criteria	Week
D1	Architecture & Database Setup	Successful N-Tier solution execution, database Migrations, and seed data initialization	W1
D2	Shopping Cart & Admin Dashboard	CRUD operations functioning in Admin Dashboard and smooth cart management	W2
D3	Payment Gateway & Order Tracking	Successful test transaction on Stripe and automated order status updating	W3
D4	Testing & Cloud Deployment	Cloud server deployment and system	W4

ID	Deliverable	Acceptance Criteria	Week
		documentation with zero open critical bugs	

1.3 Team Structure, Roles & Resource Allocation

Team Members & Roles:

- **Bassem Mohamed Allam:** Team Leader & Back-End Developer (PM & System Architecture)
- **Ahmed Mohamed Yousry:** Back-End Developer (Services & Payment Gateway)
- **Kirollos Raafat Naim:** Database & Back-End Developer (DB Design & Repository)
- **Mohamed El-Sayed Mohamed:** Database & Back-End Developer (EF Core & Migrations)
- **Habiba Ayman Fathy:** Front-End Engineer (UI/UX & Bootstrap Layout)
- **Maryse David Barty:** Front-End Engineer (Client Validation & View Styling)

RACI Matrix: *(R = Responsible, A = Accountable, C = Consulted, I = Informed)*

Activity / Role	Bassem	Ahmed	Kirollos	Mohamed	Habiba	Maryse
Project Planning & Architecture (PM)	A/R	C	C	C	I	I
Requirements Analysis (Analyst)	A/R	R	C	C	C	C
UI/UX Design & Layout	C	I	I	I	A/R	R
Database & Migrations (DB)	C	I	A/R	R	I	I
Back-End Development (Dev)	A/R	R	R	R	I	I

Activity / Role	Bassem	Ahmed	Kirollos	Mohamed	Habiba	Maryse
Quality Assurance & Testing (QA)	A	R	C	C	R	R
System Documentation (Docs)	A/R	R	R	R	R	R

1.4 Communication & Governance Schedule

Channel / Meeting	Purpose	Schedule	Responsible
Monday Status Meeting	Work distribution, blocker resolution, and progress review	Weekly (Every Monday)	Bassem Mohamed
Thursday Review Meeting	Code review, feature integration testing, and merge validation	Weekly (Every Thursday)	Bassem Mohamed & Team
GitHub PRs & Issues	Code quality checks before merging and task tracking	Continuous (All Week)	All Team Members

1.5 Timeline, Milestones & Gantt

- **Week 1:** Initial Setup, Architecture & Database Design — *Exit Criteria:* N-Tier solution setup, database creation, Identity configuration, and initial product seed data.
- **Week 2:** Shopping Cart, RBAC & Admin Dashboard — *Exit Criteria:* Full cart functionality, admin product management, and role permission enforcement.
- **Week 3:** Checkout, Stripe Payment & Delivery Tracking — *Exit Criteria:* End-to-end live payment processing via Stripe and real-time order tracking.
- **Week 4:** Polishing, Comprehensive Testing & Cloud Deployment — *Exit Criteria:* Cloud hosting deployment, QA test pass, and final documentation completion.

Visual Gantt Chart Timeline:

```
Task / Week                    | W1  | W2  | W3  | W4  |
-------------------------------+-----+-----+-----+-----|
```

Architecture & DB Setup (M1) | [X] | | | |

Cart, RBAC & Admin Dashboard (M2) | | [X] | | |

Checkout, Stripe & Tracking (M3) | | | [X] | |

Testing, Docs & Cloud Deploy (M4) | | | | [X] |

Milestones Table:

Milestone	Scope	Responsible	Target Date
M1	Architecture setup, Database creation, and ASP.NET Core MVC foundation	Kirollos & Mohamed	End of W1
M2	Session cart integration and Admin Dashboard for product management	Habiba, Maryse & Ahmed	End of W2
M3	Stripe payment gateway integration and Role-based order tracking	Bassem & Ahmed	End of W3
M4	Complete system testing, Cloud hosting deployment, and documentation	Entire Team	End of W4

1.6 Risk Register & Mitigation

ID	Risk Description	Likelihood / Impact	Mitigation Strategy	Owner
R1	Delay in Stripe Payment Gateway integration	M / H	Early initiation using test API keys and local mock payment routines	Bassem Mohamed
R2	Code conflicts during feature	H / M	Clear Git Flow adherence and	Bassem Mohamed

ID	Risk Description	Likelihood / Impact	Mitigation Strategy	Owner
	merging		daily creation of small Pull Requests	
R3	Database query performance degradation	L / H	Database Indexing, query optimization, and Eager Loading via LINQ	Kirollos Raafat
R4	Mobile responsiveness layout issues	M / M	Adoption of Bootstrap 5 grid system and continuous multi-device testing	Habiba Ayman
R5	Data loss during EF Core Migrations	L / H	Standardized Seed Data scripts and scheduled automated database backups	Mohamed El-Sayed
R6	Incomplete documentation prior to submission	M / M	Daily documentation updates synchronized with completed task milestones	Ahmed Mohamed

1.7 Success Criteria & KPIs

KPI Metric	Target Goal	Measurement Method	Evaluation Check
Catalog Search Response Time	< 1.5 Seconds	Browser Developer Tools / Lighthouse timing	M2 / M4
Checkout Success Rate	Successful Stripe Webhook processing and transaction	Automated Webhook test suites	M3

KPI Metric	Target Goal	Measurement Method	Evaluation Check
	verification		
Platform Availability Rate	⩾ **99.5% Uptime**	Monitored via Uptime Kuma / Pingdom on cloud hosting	M4
Responsive Design Compliance	100% Viewport Compatibility	Chrome Responsive Viewer across devices	M2 / M4
Requirement Coverage	100% Core Functional Implementation	End-to-end Acceptance Testing	M4
Schedule Adherence	⩾ **95% Milestone Deliverable Timeliness**	Milestone review logs and submission tracking	M1 - M4

Task 2 — Literature Review & Technical Foundation

2.1 Literature Review & Architectural Approaches

Modern E-commerce platforms like Amazon, B.TECH, and Jumia rely on structured architectural patterns to achieve high availability, fault tolerance, and responsiveness under scale. Below is a comparative review of real-world enterprise techniques:

- **B.TECH E-Commerce Architecture:** Utilizes Response / In-Memory Caching & Pagination for high-traffic browsing and search filtering. Note: Cart state persistence relies on server database storage for logged-in users rather than purely session-based volatile storage to avoid losing items across sessions.
- **Decoupled N-Tier Architecture:** Enforces clear separation between Presentation, Business Logic Layer (BLL), and Data Access Layer (DAL) to minimize tight coupling and enable independent component updates.

2.2 System Architecture & Unit of Work Pattern

TechXpress follows a 4-Tier Enterprise Architecture leveraging ASP.NET Core MVC, Entity Framework Core, Repository Pattern, and Unit of Work (UoW):

- **Presentation Layer:** ASP.NET Core MVC Views, Controllers, and ViewModels with Bootstrap 5 dynamic rendering.

- **Business Logic Layer (BLL):** Services handling cart checkout, role authorization, dynamic search filters, and external payment integration.
- **Data Access Layer (DAL) & Unit of Work:** Generic Repositories encapsulated under a Unit of Work interface to commit local database operations atomically within ASP.NET Core.
- **Unit of Work & Stripe Decoupling:** Database transactions managed by Unit of Work strictly cover local database entities (Inventory updates & Order record creation in ASP.NET Core). External Stripe payment confirmation cannot participate directly in ACID database transactions; therefore, payment verification is decoupled and processed asynchronously via Stripe Webhooks with Idempotency keys to guarantee data consistency.

2.3 Technology Stack & Justification

Technology	Version	Justification / Rationale
ASP.NET Core MVC	.NET 8.0	High-performance, secure enterprise framework providing robust MVC architecture for separation of concerns.
Microsoft SQL Server	SQL Server 2022	Reliable relational database management system ensuring data integrity and secure complex transaction handling.
Entity Framework Core	EF Core 8.0	Industry-standard ORM supporting Code-First design, automated Migrations, and SQL-injection-safe LINQ queries.
ASP.NET Core Identity	8.0	Built-in secure identity management framework for authentication, password hashing, and Role-Based Access Control.
Stripe SDK	v43+	Globally recognized, secure, PCI-DSS compliant payment gateway handling encrypted financial transactions.

Technology	Version	Justification / Rationale
Front-End Stack	HTML5, CSS3, Bootstrap 5, JS	Lightweight, modern responsive front-end libraries ensuring fast load times across all mobile and desktop devices.

Task 3 — System Requirements & Analysis

3.1 User Stories & Functional Requirements Mapping

User Stories are mapped directly to corresponding Functional Requirements (FRs) to maintain full traceability across system boundaries:

- **US1 (Dynamic Product Search):** As a Customer, I want to search and filter products by category and price with a response time strictly under 1.5 seconds so that I can find electronics quickly. [Priority: Must | Mapped FR: FR2]
- **US2 (Shopping Cart Management):** As a Customer, I want to add/remove items and update quantities in my shopping cart so that I can prepare my purchase. [Priority: Must | Mapped FR: FR3]
- **US3 (Secure Stripe Checkout):** As a Customer, I want to complete my checkout using Stripe payment gateway so that my payment is handled securely. [Priority: Must | Mapped FR: FR3, FR4]
- **US4 (Admin Dashboard & Inventory):** As an Admin, I want to manage product listings, pricing, and stock levels through a DataTables dashboard. [Priority: Must | Mapped FR: FR5]
- **US5 (Order Status Tracking):** As a Customer/Delivery agent, I want to track and update order dispatch status in real time. [Priority: Should | Mapped FR: FR4, FR5]

3.2 Functional Requirements (FRs)

- **FR1 (Authentication & RBAC):** The system shall authenticate users and assign roles (Customer, Admin, Delivery) using ASP.NET Identity with secure password hashing.
- **FR2 (Catalog & Dynamic Search):** The system shall provide dynamic search, category filtering, price sorting, and pagination with response times strictly < 1.5s.
- **FR3 (Cart & Order Processing):** The system shall persist shopping cart items and calculate order subtotals, taxes, and shipping fees.
- **FR4 (Stripe Payment Integration):** The system shall process online payments securely via Stripe API and verify transactions using asynchronous Stripe Webhooks with idempotency keys.

- **FR5 (Admin Dashboard Management):** The system shall allow Admins to perform complete CRUD operations on products, categories, and order statuses via interactive DataTables.

3.3 Non-Functional Requirements (NFRs)

- **NFR1 (Performance):** Catalog browsing and search query execution time shall not exceed 1.5 seconds under standard concurrency.
- **NFR2 (Availability):** The platform shall maintain 99.5% uptime on Cloud hosting environments (Azure App Service / Somee IIS).
- **NFR3 (Security):** All communications must enforce HTTPS/TLS 1.3, and all input parameters must be sanitized against OWASP Top 10 vulnerabilities (CSRF, XSS, SQL Injection).

3.5 Use Case Descriptions

- **UC-01 (Browse & Search Catalog):**
  - *Actor:* Customer
  - *Preconditions:* User is on the storefront home page.
  - *Main Flow:* Customer enters search keywords or selects category filters; system queries database and renders filtered product catalog within 1.5s.
- **UC-02 (Checkout with Stripe):**
  - *Actor:* Customer
  - *Preconditions:* Customer has items in the shopping cart and is authenticated.
  - *Main Flow:* Customer clicks checkout, enters shipping details, and is redirected to Stripe secure payment page. Upon successful payment, Stripe Webhook updates the order status in the local database and deducts stock inventory.
- **UC-03 (Manage Inventory):**
  - *Actor:* Admin
  - *Preconditions:* User is authenticated with Administrator role.
  - *Main Flow:* Admin accesses the DataTables dashboard, views current product listings, and performs create, update, or delete operations on product inventories.

3.6 Assumptions & Constraints

- **Assumption 1:** Customers have stable internet connectivity and modern Web browsers supporting JavaScript execution and CSS3 styling.
- **Constraint 1 (Cloud Hosting):** System deployment is restricted to cloud web hosting environments specifically Azure App Service or Somee IIS environment.
- **Constraint 2 (Framework & Architecture):** Core server application must be built exclusively using ASP.NET Core MVC adhering to N-Tier Architecture and Entity Framework Core.